bigdata_report.qmd

Workflow:
I- Importing the dataset: In the first step of my analysis I imported the TSV dataset
II- Data Analysis: I observed the data set, i noticed that it is composed of 30 predictors(categorical and numerical) and 1 continuous outcome(bone_mineral_density_score). I chose Regression Model which is a supervised Machine Learning model to conduct my analysis.
III- Models chosen:
Within the category of Regression model, i chose the following three models to conduct my analysis:

- Linear Regression: To explore and quantify the linear relationship between the predictor(s) and a continuous outcome variable.
- Random Forest: Used to identify intrinsic correlations between predictors and the outcome through an ensemble of decision trees
- K-Nearest Neighbours: Used based on the principle of neighborhood decisions, predicting outcomes by considering the closest similar observations in the dataset.

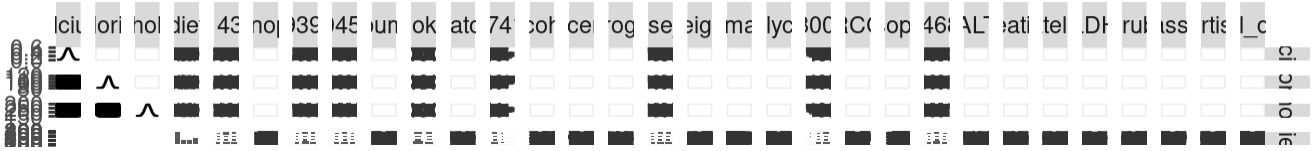
```
# Import the libraries
library(knitr)
library(tidyverse)
library(tidymodels)
library(GGally)
library(vip)
```

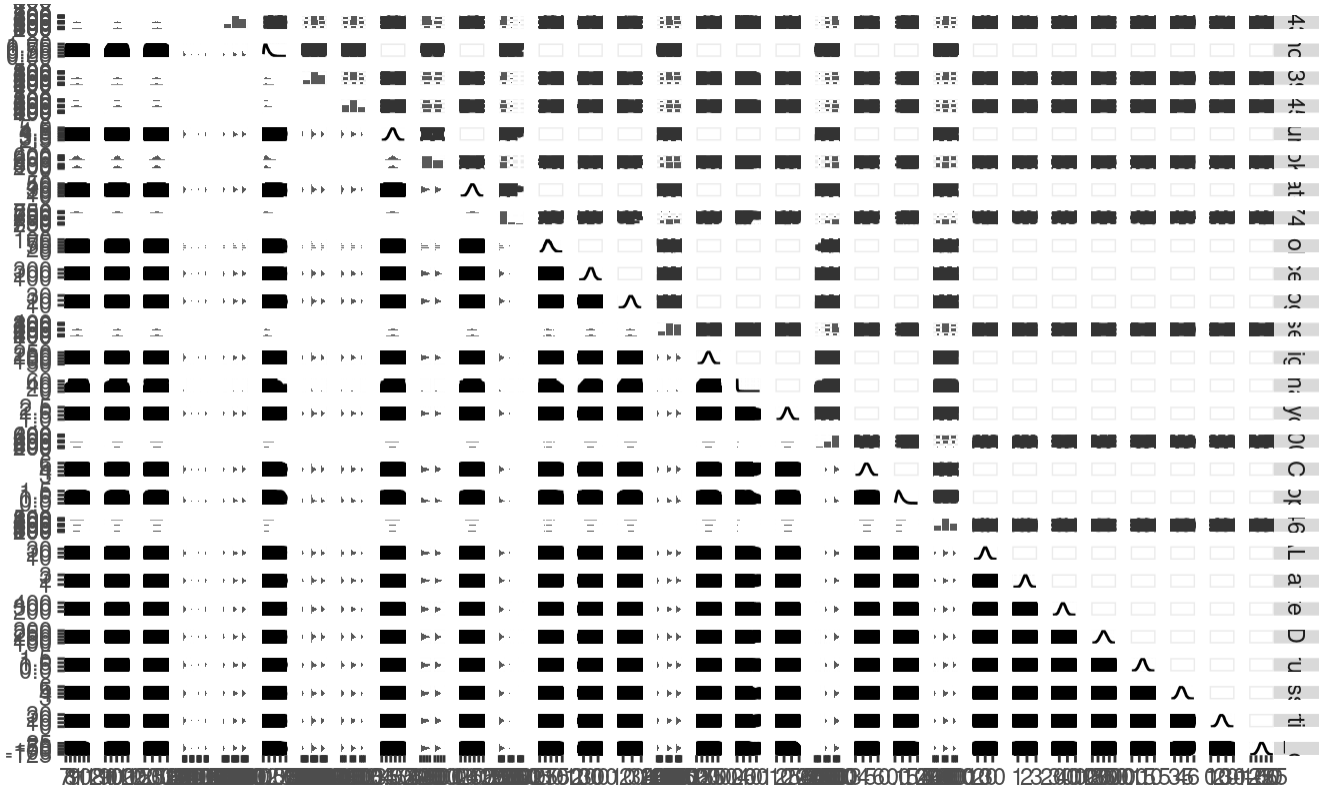
```
# Load the dataset
bone_density_data <- read_tsv("~/bigdata_report2/bone_mineral_density_study.tsv")
```

```
Rows: 10000 Columns: 31
── Column specification ────────────────────────────────────────────────────────
Delimiter: "\t"
chr  (9): diet, rs1143627, rs9939609, rs1045642, smoking, rs7412, exercise_l...
dbl (22): calcium, chloride, total_cholesterol, eosinophils, albumin, melato...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Visualise the relationships between the variables
scatterplot_matrix <- ggpairs(bone_density_data)
print(scatterplot_matrix)
```





With such a large dataset (10,000 rows × 31 columns), the scatter plot matrix generated is not clear. Since there are 31 variables, the matrix would try to display 31×31=961 small plots in a single figure, which is too dense for effective visualization. In addition, with 10,000 data points per plot, points overlap heavily, making the scatter plots in the matrix difficult to interpret. In this report, I will focus on exploring the most influential predictors on the outcome and generate scatter plots including only these important variables. This will allow me to generate a scatter plot that is much clearer and easier to interpret.

```
# Split the data into training and testing
set.seed(123)
bone_density_data_split <- initial_split(bone_density_data, prop = 0.75)
bone_density_data_training <- training(bone_density_data_split)
bone_density_data_testing <- testing(bone_density_data_split)
cv_folds <- vfold_cv(bone_density_data_training, v = 5)
```

- **I-Model 1:Linear Regression**
  For the purpose of implementing Linear Regression, first i have to define the model by defining its type and set the engine which is the computational back end of the model.
  Then, I created the recipe, defining the outcome variable (bone_mineral_density_score) and the predictors in the dataset. I used the step_dummy() function to convert categorical predictors into numeric dummy variables, making them suitable for models like linear regression. I also applied step_normalize() to normalize all numeric predictor variables, so that each has a mean of 0 and a standard deviation of 1. Normalization ensures that the predictors have the same scale.
  Then i created the workflow that includes the model type with its engine and the recipe in a single object.
  To optimize the predictive performance of the linear regression model, I performed hyperparameter

To optimize the predictive performance of the linear regression model, I performed hyperparameter tuning. I defined the grid of combinations of penalty and mixture values (3 values each) that will be tested during hyperparameter tuning to find the best model. Using cross-validation, I evaluated all parameter combinations to determine their effect on model performance, measured by RMSE and R^2. The best-performing model, selected based on the lowest RMSE, had penalty = 0 and mixture = 0.05, achieving an RMSE of 0.447 and an R^2 of 0.999998, indicating excellent predictive accuracy.

```
#Define the model (with tuning parameters)

lm_model <- linear_reg(
  penalty = tune(),   # regularization strength
  mixture = tune()    # elastic net mixing parameter
) %>%
  set_engine("glmnet")


#Create the recipe

lm_recipe <-
  recipe(bone_mineral_density_score ~ .,
         data = bone_density_data_training) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_predictors())


#Create the workflow
lm_workflow <- workflow() %>%
  add_model(lm_model) %>%
  add_recipe(lm_recipe)

#Define tuning grid

lm_tuning_grid <- grid_regular(
  penalty(),
  mixture(range = c(0.05, 1.00)),
  levels = 3
)

#Run tuning

lm_tuning_results <- lm_workflow %>%
  tune_grid(
    resamples = cv_folds,
    grid = lm_tuning_grid
  )

#Collect metrics & select best parameters
tuning_metrics <- lm_tuning_results %>% collect_metrics()
best_params <- lm_tuning_results %>% select_best("rmse")
```

```
# Show both rmse & rsq for the best model
best_metrics <- tuning_metrics %>%
  filter(.config == best_params$.config)

# Show both rmse & rsq for the best config
best_metrics <- tuning_metrics %>%
  filter(.config == best_params$.config)
kable(best_metrics)
```

penalty	mixture	.metric	.estimator	mean	n	std_err	.config
0	0.05	rmse	standard	0.4471842	5	0.0058113	Preprocessor1_Model1
0	0.05	rsq	standard	0.9999981	5	0.0000002	Preprocessor1_Model1

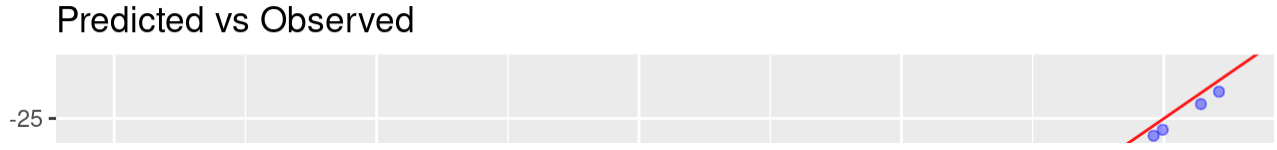
I then fitted this model on the training subset, and then used this fitted model to do predictions on the testing subset.
Then I used the fitted model to do predictions on the testing subset. The red line represents the line of perfect prediction (y = x), with slope 1 and intercept 0, which represents a reference for ideal predictions. Each blue point corresponds to one observation, showing the relationship between the observed bone mineral density score (x-axis) and the predicted value (y-axis). A point that falls on the red line shows that the predicted value matches the observed value exactly.

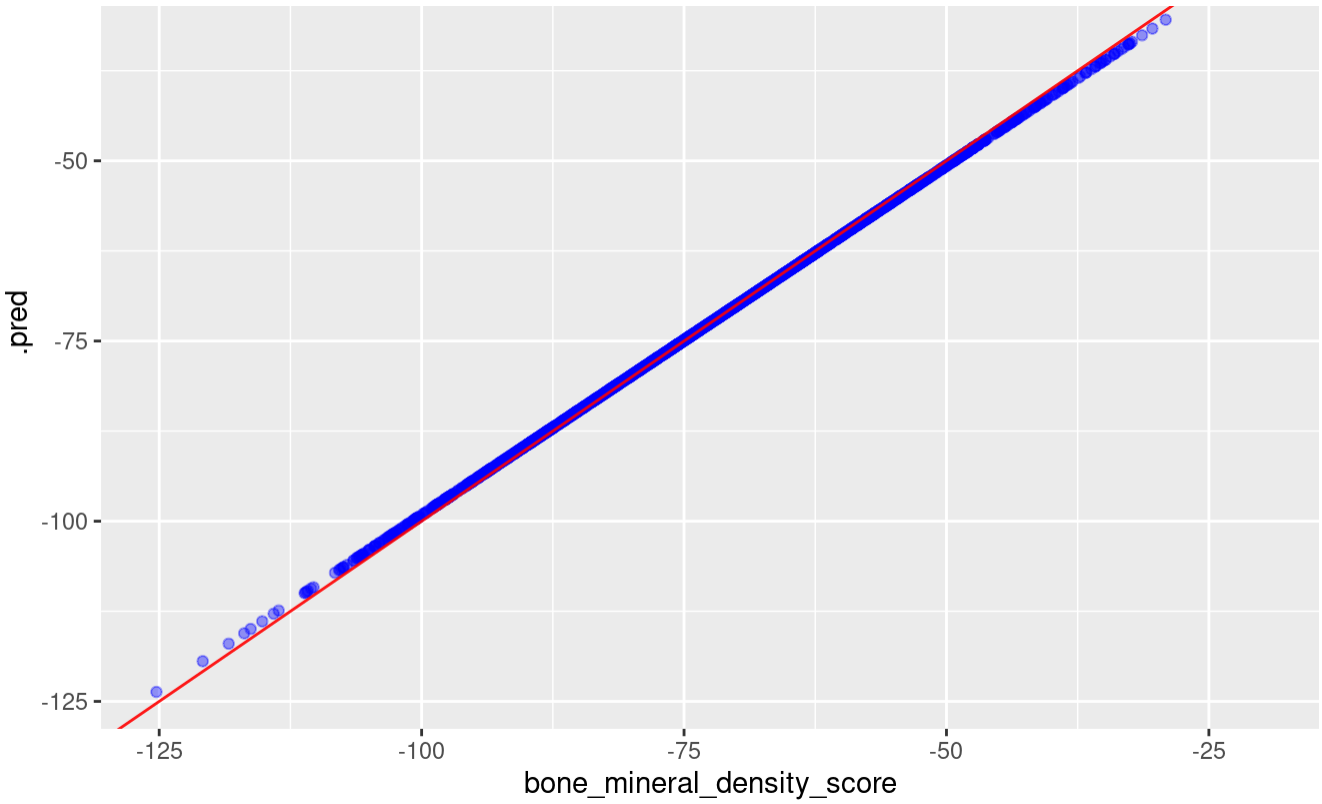
```
#Finalize workflow with best parameters
lm_final_wf <- lm_workflow %>%
  finalize_workflow(best_params)

# Fit on training data
lm_final_fit <- lm_final_wf %>%
  fit(data = bone_density_data_training)


#Predictions on test set

lm_wf_prediction <- lm_final_fit %>%
  predict(bone_density_data_testing) %>%
  bind_cols(bone_density_data_testing)
#Visualization of predictions
lm_wf_prediction %>%
  ggplot(aes(x = bone_mineral_density_score, y = .pred)) +
  geom_point(alpha = 0.4, colour = "blue") +
  geom_abline(colour = "red", alpha = 0.9, intercept = 0, slope = 1) +
  labs(title = "Predicted vs Observed")
```





In my analysis, most points lie on the red line. This indicates that the linear regression model captures the linear relationship between the outcome variable and the predicted values very well, suggesting a strong linear relationship. However there are few blue points that deviates from the line. In addition both the x-axis and the y-axis ranges from -125 to +25 which ensures that the plot uses a consistent scale, allowing a clear and accurate visual comparison between observed and predicted values.

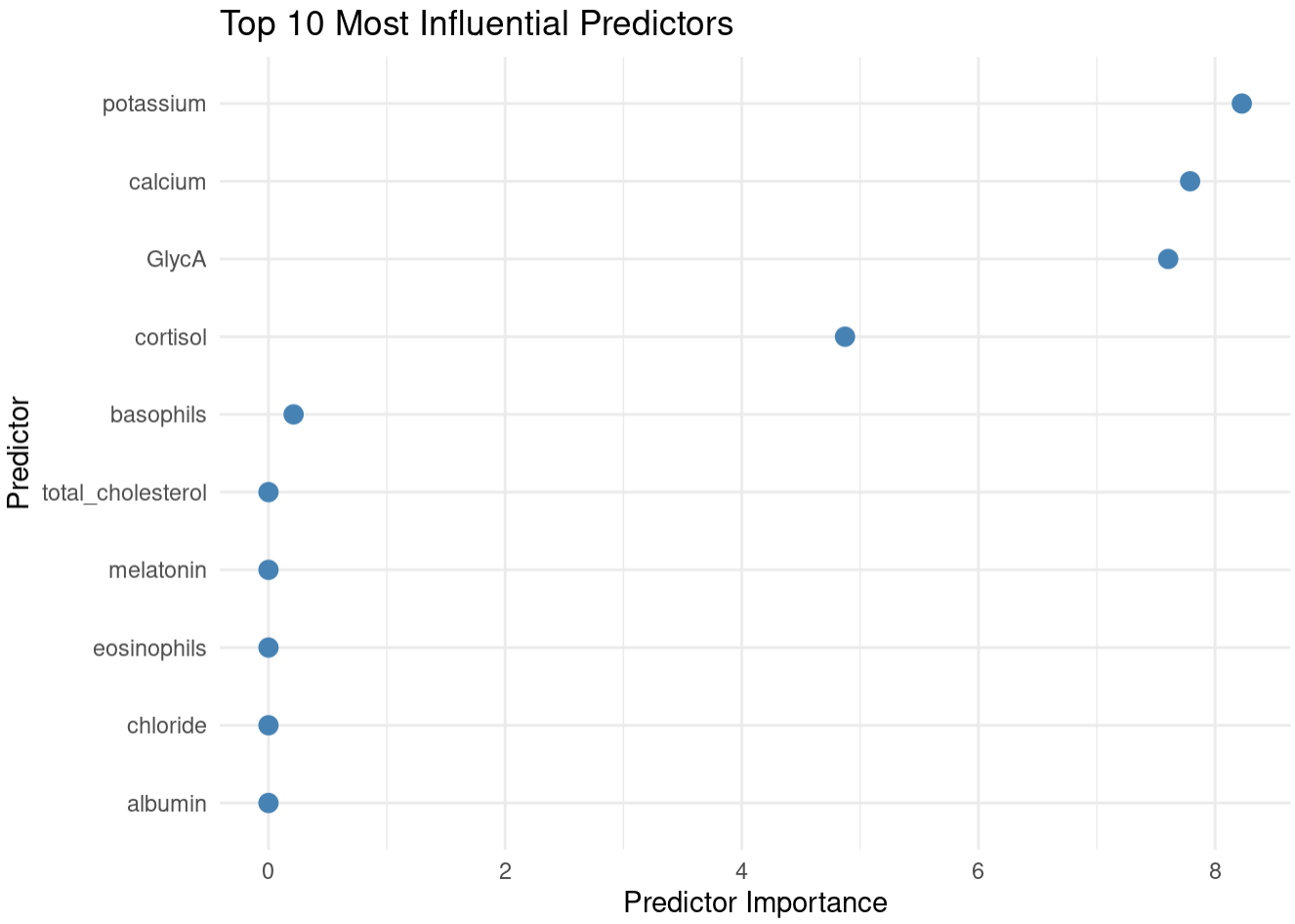
```
#Check the prediction metrics
lm_wf_prediction %>%
  metrics(truth = bone_mineral_density_score, estimate = .pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard       0.459
2 rsq     standard       1.00
3 mae     standard       0.367
```

These metrics (rmse, rsq, and mae) reflect the performance of the model in prediction.
MAE(mean absolute error) is the average magnitude of the error between predictiom and the outcome(the closer this metrics to 0 the lower the error)
RMSE(root mean squared error) is the same concept as MAE but more weigh on higher error(the closer this metrics to 0 the lower the error)
RSQ(r-squared score) R-squared (R^2) is a metric that shows how much of the variability in the outcome (dependent variable) is explained by the model's predictors.
The values obtained suggest that the model captures the relationship between the predictor(s) and the outcome very well, with minimal prediction error.

```
# Extract coefficients (top 10 by importance)
lm_coefficients <- lm_final_fit %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  filter(term != "(Intercept)") %>%
  mutate(importance = abs(estimate)) %>%
  arrange(desc(importance)) %>%
  slice_head(n = 10)

#  Dot plot of predictor importance
lm_coefficients %>%
  ggplot(aes(x = importance, y = fct_reorder(term, importance))) +
  geom_point(size = 3, colour = "steelblue") +
  labs(
    x = "Predictor Importance",
    y = "Predictor",
    title = "Top 10 Most Influential Predictors"
  ) +
  theme_minimal()
```



From the plot above, we can see that Potassium has the highest importance score among the predictor variables.

- **II- Model2: Random Forest**

I performed hyperparameter tuning — the process of training multiple models on resampled datasets to evaluate their performance and select the best parameter settings — on the Random Forest model to optimize its predictive performance.
I did tuning for the following parameters:
mtry: Adjusting the number of predictors sampled at each split.
trees: the total number of trees
min_n: The minimum number of observations required at a terminal node, in order to optimize model performance
Same as Linear Regression i set up the structure of Linear Regression and defined the recipe and workflow that includes the model type with its engine and the recipe in a single object.
Then I defined a tuning grid for the Random Forest hyperparameters, then performed hyperparameter tuning to evaluate different parameter combinations and selected the best-performing parameters based on RMSE value.
Finally, I used the model with these optimal parameters and fitted it on the training subset to build the final predictive model.

```
# Define the random forest model
rf_model_tune <- rand_forest(
  mtry  = tune(),
  trees = tune(),
  min_n = tune()
) %>%
  set_mode("regression") %>%
  set_engine("ranger", importance = "permutation")

# Define the recipe
rf_recipe <-
  recipe(bone_mineral_density_score ~ ., data = bone_density_data_training) %>%
  step_dummy(all_nominal_predictors())
  # step_normalize() removed – RF does not need normalization

# Combine model and recipe into a workflow
rf_tuning_workflow <- workflow() %>%
  add_model(rf_model_tune) %>%
  add_recipe(rf_recipe)

# Define a lighter tuning grid
set.seed(123)
rf_tuning_grid <- grid_latin_hypercube(
  finalize(mtry(), bone_density_data_training),  # ensure valid range
  trees(range = c(200L, 800L)),                  # smaller tree range
  min_n(range = c(2L, 10L)),                     # reasonable min_n
  size = 10                                      # 10 configs only
)
```

```
# Tuning results
rf_tuning_results <- rf_tuning_workflow %>%
  tune_grid(
    resamples = cv_folds,
    grid = rf_tuning_grid,
    control = control_grid(save_pred = TRUE, verbose = TRUE)
  )
```

```
# Collect metrics
tuning_metrics <- rf_tuning_results %>%
  collect_metrics()

# Select the best tuning parameters (based on RMSE)
rf_tuning_best_params <- rf_tuning_results %>%
  select_best("rmse")

best_config <- rf_tuning_best_params$.config

best_with_metrics <- rf_tuning_results %>%
  collect_metrics() %>%
  filter(.config == best_config)

kable(best_with_metrics)
```

mtry	trees	min_n	.metric	.estimator	mean	n	std_err	.config
31	432	3	rmse	standard	2.7803559	5	0.1063010	Preprocessor1_Model05
31	432	3	rsq	standard	0.9709715	5	0.0019946	Preprocessor1_Model05

The best-performing model with RMSE = 2.78and R^2 = 0.98 on the training data.

Number of trees equal 432 is high and reduces the variance of the model, and decreases the value of RMSE thus decreasing the prediction error. mtry value equal 9, means that the model selects 9 predictors at each slit. A low value of min_n suggests that the trees to grow deeper so the tree can split down to very small groups of data.

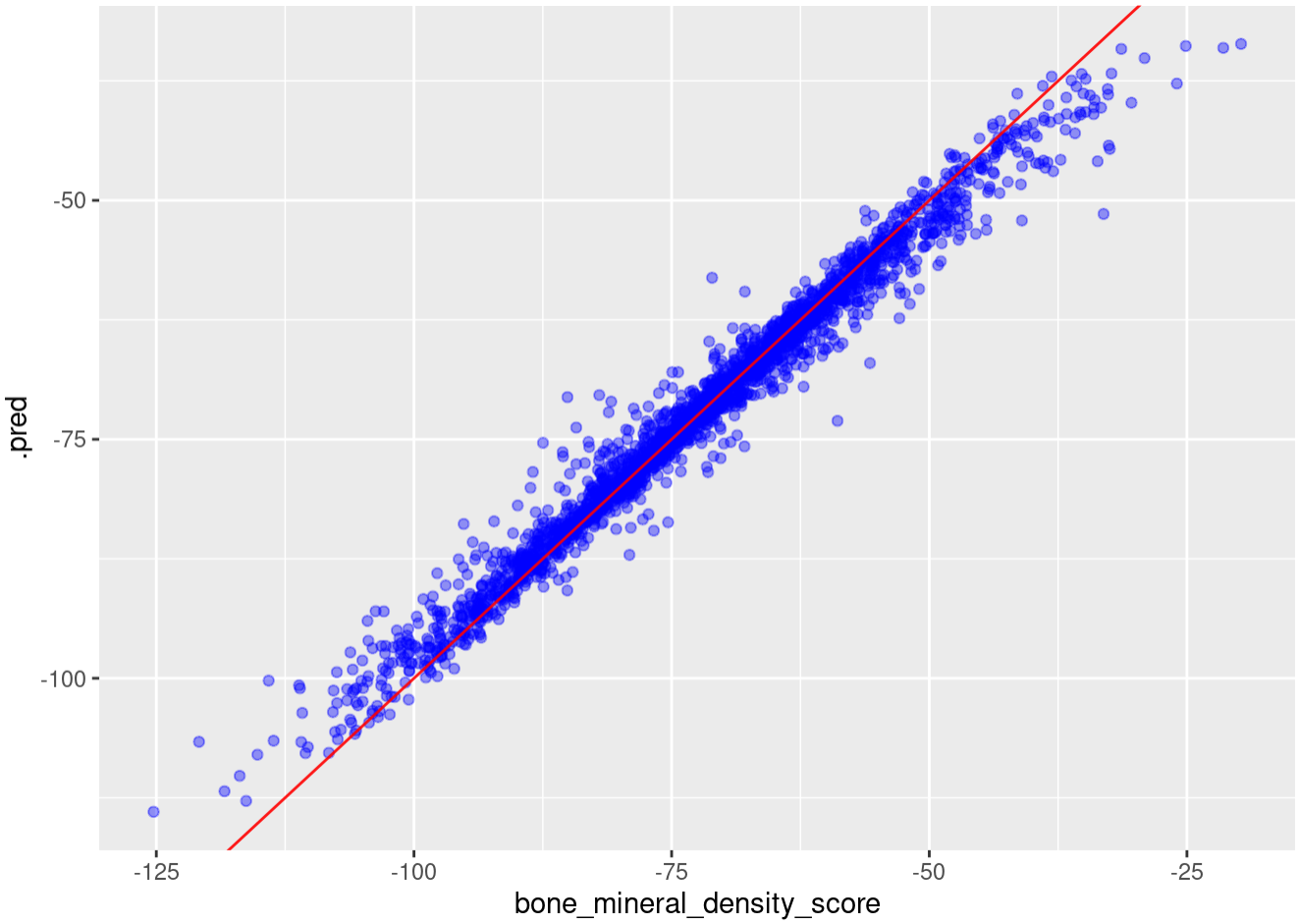
```
#Finalize workflow with best parameters
rf_final_workflow <- rf_tuning_workflow %>%
  finalize_workflow(rf_tuning_best_params)

#Fit final model on training data
rf_final_fit <- rf_final_workflow %>%
  fit(data = bone_density_data_training)
```

Then I used the fitted model to do predictions on the testing subset

```
rf_wf_prediction <- rf_final_fit %>%
  predict(bone_density_data_testing) %>%
  bind_cols(bone_density_data_testing)

rf_wf_prediction %>%
  ggplot(aes(x=bone_mineral_density_score, y=.pred))+
  geom_point(alpha = 0.4, colour = "blue")+
  geom_abline(colour = "red", alpha = 0.9)
```



The red line represents perfect prediction, while the blue points show the relationship between the predicted and actual values. Most of the blue points are closely clustered along the red line, and those that deviate are still near it, indicating low residual errors. A few points are farther from the line, but they are very few. Overall, the model makes good predictions, although its performance is slightly lower compared to linear regression.

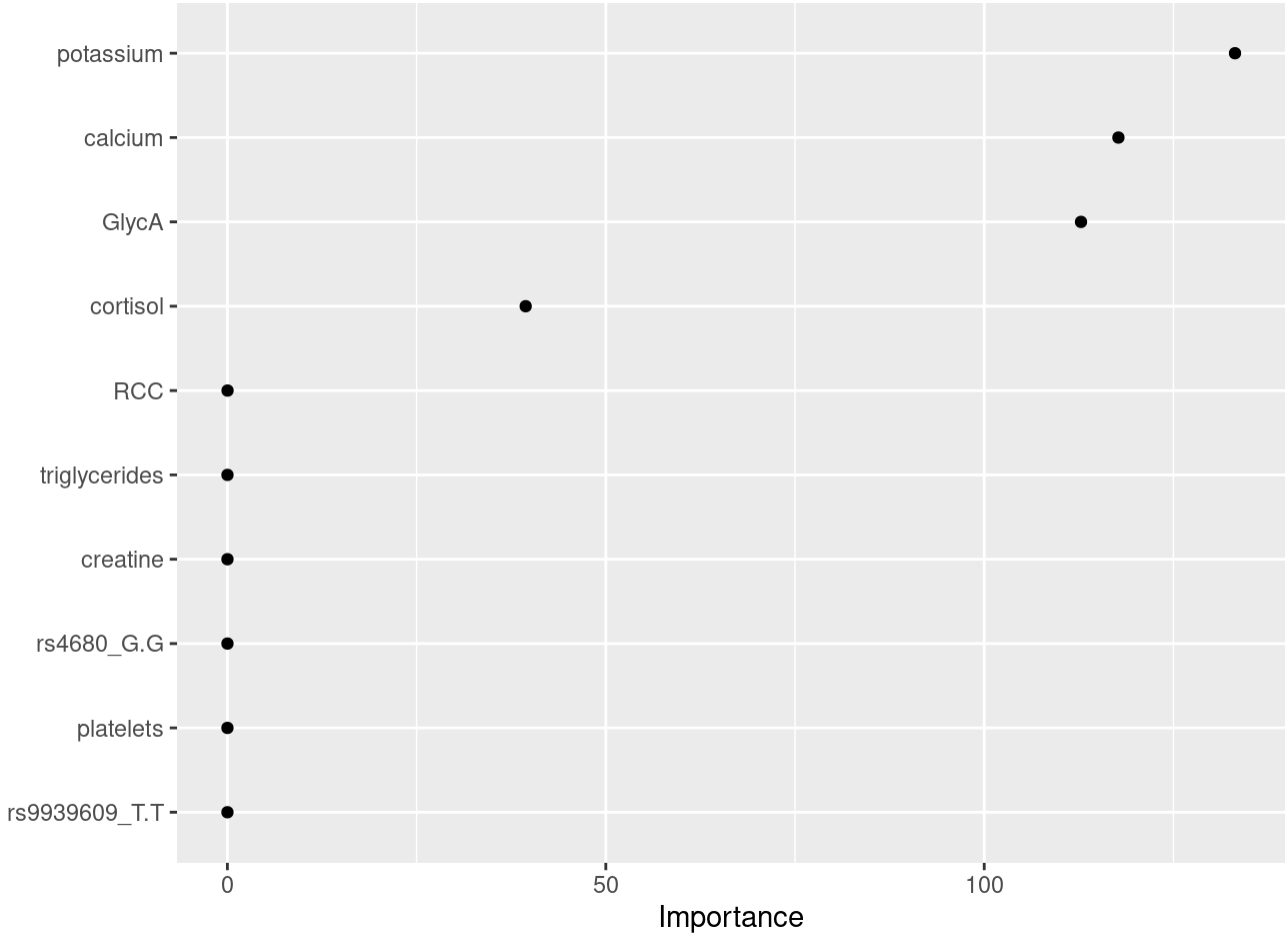
```
rf_wf_prediction %>%
  metrics(truth = bone_mineral_density_score, estimate = .pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard       2.63
```

```
1 rmse      standard       2.05
2 rsq       standard       0.975
3 mae       standard       1.74
```

The values obtained suggest that the model captures the relationship between the predictor(s) and the outcome very well, with minimal prediction error.

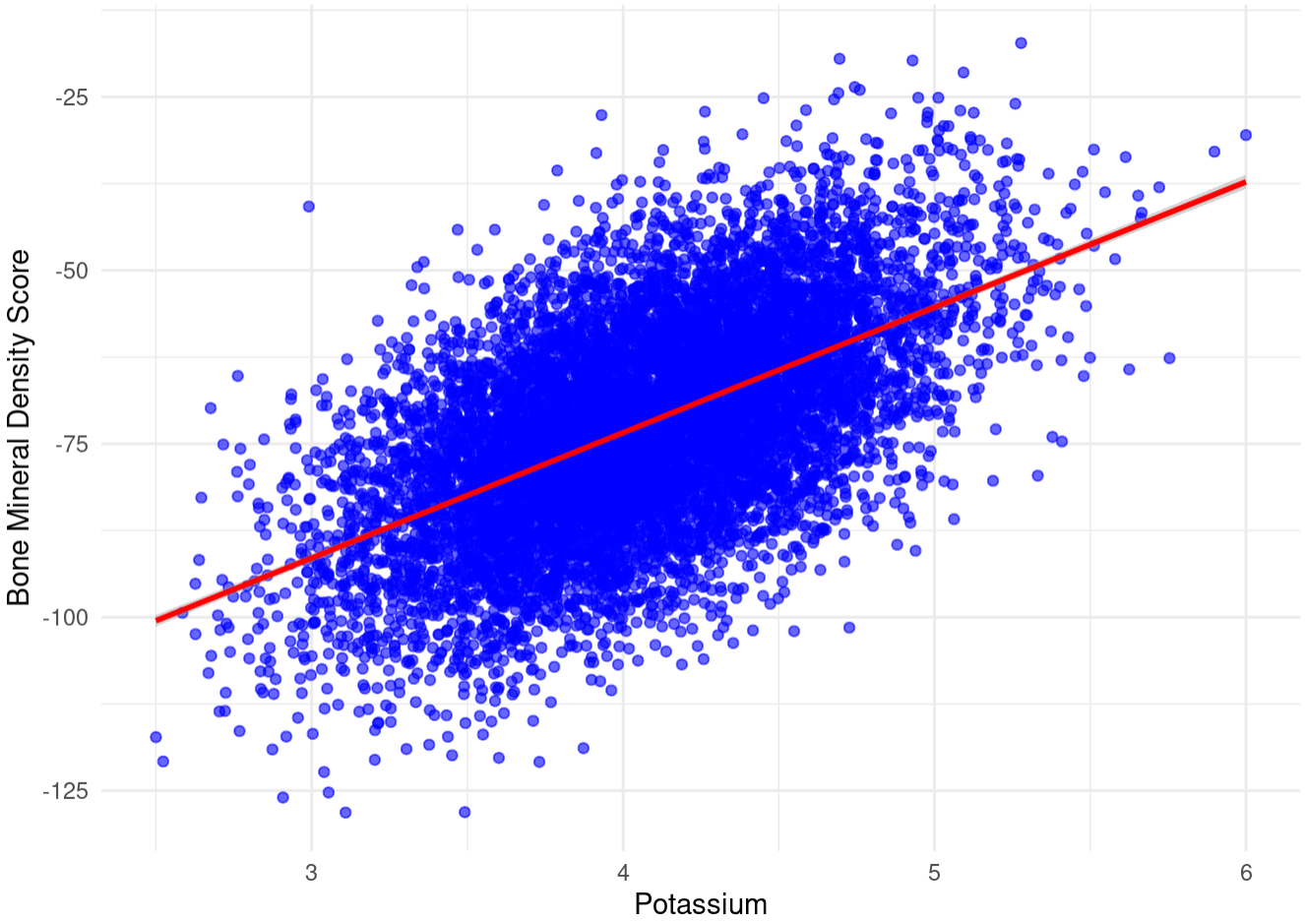
```
# Explainability check - variables importance plot
rf_final_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 10, geom = "point")
```



The importance plot obtained is equivalent to the one obtained in the linear regression. Potassium is the most important predictor.
I generated a scatter plot and performed pearson correlation to study the correlation between potassium and the Bone Mineral Density Score.

```
# Data exploration - scatter plot to visualise the relationship between potassium and the outco
ggplot(bone_density_data, aes(x = potassium, y = bone_mineral_density_score)) +
  geom_point(alpha = 0.6, color = "blue") +
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(x = "Potassium", y = "Bone Mineral Density Score") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```



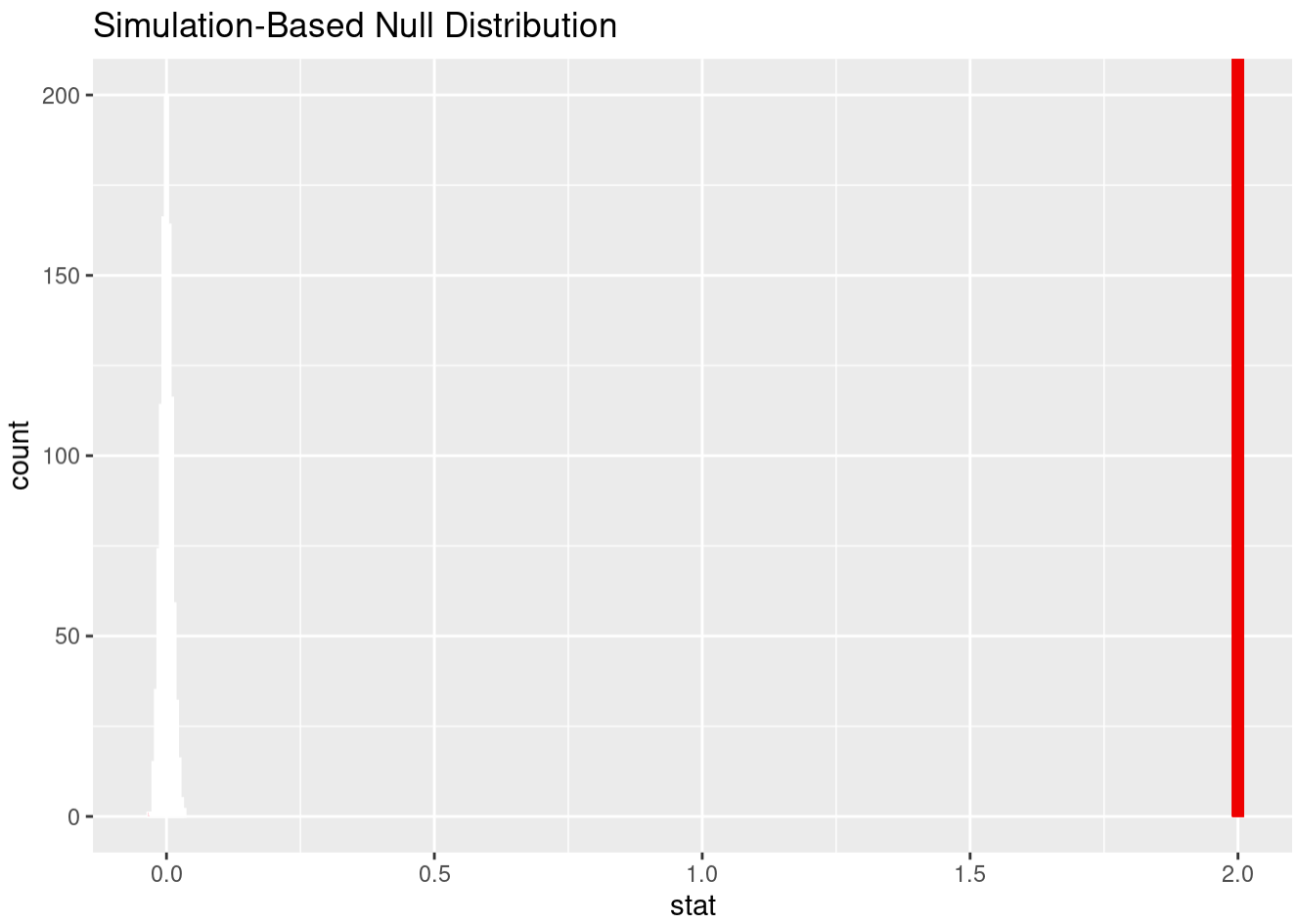

```
#Pearson correlation
bone_density_data_correlation_observed <- bone_density_data %>%
  specify(potassium ~ bone_mineral_density_score) %>%
  calculate(stat = "correlation") %>%
  dev.off()


bone_density_data_correlation_observed
```

```
png
  2
```

```
bone_density_data_correlation_null <- bone_density_data %>%
  specify(potassium ~ bone_mineral_density_score) %>%
  hypothesize(null = "independence") %>%
  generate(reps = 1000, type = "permute") %>%
  calculate(stat = "correlation")

visualize(bone_density_data_correlation_null) +
  shade_p_value(obs_stat = bone_density_data_correlation_observed, direction = "two-sided")
```



The potassium predictor shows positive correlation with the bone mineral density score, having correlation coefficient of +1.

```
corr_pval = bone_density_data_correlation_null %>%
  get_p_value(obs_stat = bone_density_data_correlation_observed, direction = "two-sided")
```

```
Warning: Please be cautious in reporting a p-value of 0. This result is an
approximation based on the number of `reps` chosen in the `generate()` step.
See `?get_p_value()` for more information.
```

```
corr_pval
```

```
# A tibble: 1 × 1
  p_value
    <dbl>
1       0
```

The p-value equal 0 shows that the correlation result obtained is statistically significant
Recent studies support the positive role of potassium in bone health. Higher dietary potassium intake has been linked to increased bone mineral density and a lower risk of osteoporosis.

- **III- K NEAREST NEIGHBOURS**

- **III- K-NEAREST NEIGHBOURS**

For the KNN regression model, the main hyperparameter is k (the number of neighbors to be taken into consideration when making a decision), which determines how many of the closest observations are used to predict the outcome for a new sample. In this study, the dataset includes 30 predictors, and to capture an appropriate balance between overfitting and generalization, the tuning grid was set to evaluate k values from 1 to 10. This range provides a comprehensive yet computationally efficient search space for identifying the optimal neighborhood size for the purpose of doing a prediction for an observation.

```
# Define the model - tuning of the "neighbours" hyperparameter
knn_model <-
  nearest_neighbor(
    neighbors = tune(),
    weight_func = "triangular") %>%
  set_mode("regression") %>%
  set_engine("kknn")

# Set up the tuning grid
knn_tuning_grid <- grid_regular(
  neighbors(range = c(1L, 10L))
)
```

I then created the recipe, same as in the 2 previous models.

```
# Create the recipe
knn_recipe <- recipe(bone_mineral_density_score ~ .,
                     data = bone_density_data_training) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_predictors())
```

Then i created the workflow that contain the model structure and the recipe. Then, I performed hyperparameter tuning by evaluating the defined grid of neighbor values using cross-validation (cv_folds). This allowed me to systematically assess model performance across different k values.

```
# Create the workflow
knn_wf <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(knn_recipe)

# Tune
knn_tuning_results <- knn_wf %>%
  tune_grid(
    resamples = cv_folds,
    grid = knn_tuning_grid
  )
```

After tuning, I collected the performance metrics (RMSE, R^2) for each candidate number of neighbors.

These results allow me to compare models and select the optimal k.

```
# Visualise tuning metrics
knn_tuning_results %>%
  collect_metrics()
```

```
# A tibble: 6 × 7
  neighbors .metric .estimator   mean     n std_err .config
      <int> <chr>   <chr>       <dbl> <int>   <dbl> <chr>
1         1 rmse    standard   14.3       5 0.158   Preprocessor1_Model1
2         1 rsq     standard    0.220     5 0.00448 Preprocessor1_Model1
3         5 rmse    standard   11.2       5 0.112   Preprocessor1_Model2
4         5 rsq     standard    0.435     5 0.00415 Preprocessor1_Model2
5        10 rmse    standard   10.6       5 0.121   Preprocessor1_Model3
6        10 rsq     standard    0.560     5 0.00569 Preprocessor1_Model3
```

I then extracted the best hyperparameter based on the RMSE performance metric, this model is then used for fitting on the training subset.

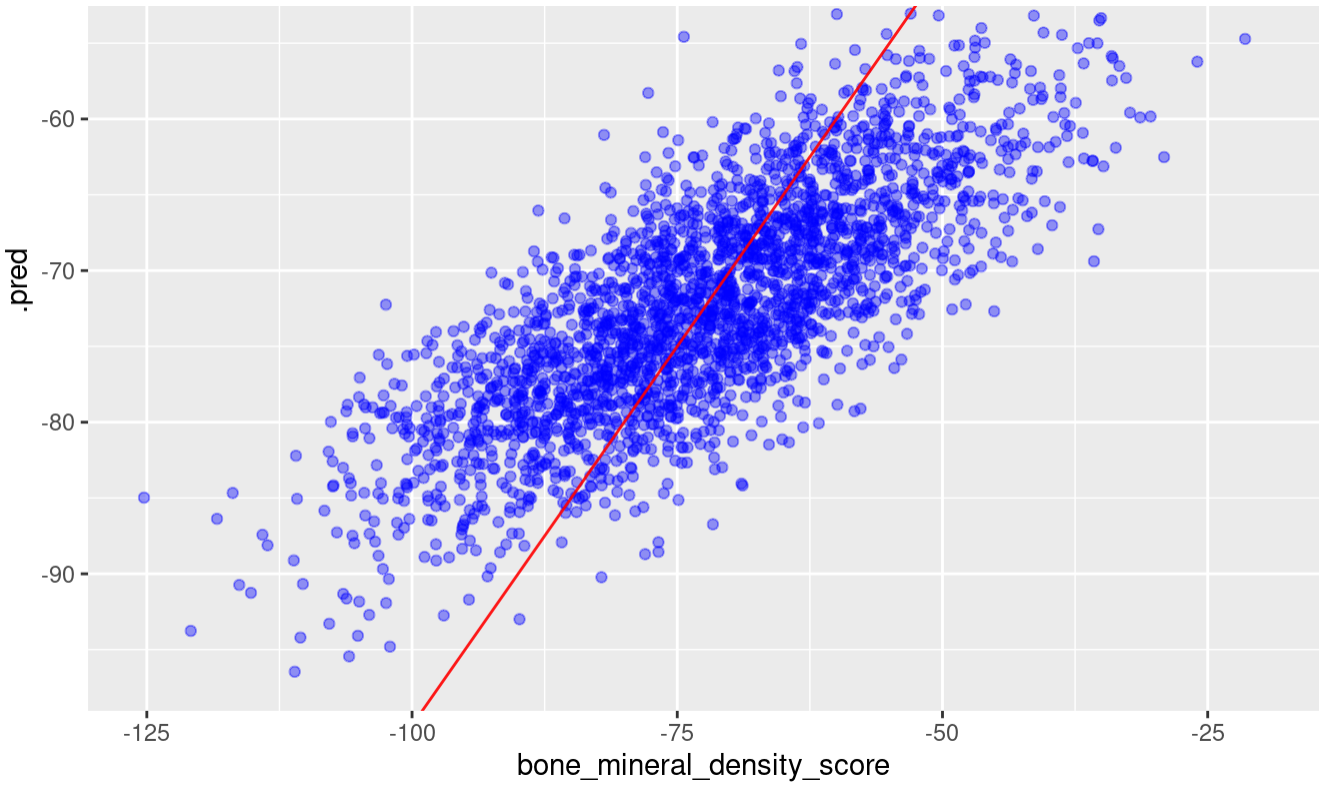
```
# Extract best hyperparameters based on the rmse
knn_tuning_best_parameters  <- knn_tuning_results %>%
  select_best("rmse")
```

The best model obtained after tuning has number of neigbors equal 10. So for a given observation(sample) 10 neighbors. So for any given observation, the algorithm considers the 10 closest neighbors in the data when making a prediction. Then i used this model to do fitting on the training subset.

```
# Finalise the workflow
final_knn_wf <- knn_wf %>%
  finalize_workflow(knn_tuning_best_parameters)

# Fit the workflow with the best hyperparameters
final_knn_fit <- final_knn_wf %>%
  last_fit(bone_density_data_split)
```

```
# Visualise the prediction
final_knn_fit %>%
  collect_predictions() %>%
  ggplot(aes(x=bone_mineral_density_score, y=.pred))+
  geom_point(alpha = 0.4, colour = "blue")+
  geom_abline(colour = "red", alpha = 0.9)
```



In this model, the points are more dispersed from the red line compared to the previous two models, indicating larger residual errors; therefore, the KNN model shows the lowest predictive performance.

```
# Collect the final metrics
final_knn_fit %>%
  collect_metrics()
```

```
# A tibble: 2 × 4
  .metric .estimator .estimate .config
  <chr>   <chr>          <dbl> <chr>
1 rmse    standard      10.5   Preprocessor1_Model1
2 rsq     standard       0.603 Preprocessor1_Model1
```

The high RMSE value and the RSQ values being further from 1 in the KNN model further validate the model lower performance compared to the previous 2 models(Linear regression and Random forest)

IV- Model Comparison

```
library(dplyr)
library(tidymodels)

# Linear model metrics
lm_metrics <- lm_wf_prediction %>%
  metrics(truth = bone_mineral_density_score, estimate = .pred) %>%
  mutate(Model = "Linear Model")

# Random Forest metrics
rf_metrics <- rf_wf_prediction %>%
```

```
  metrics(truth = bone_mineral_density_score, estimate = .pred) %>%
  mutate(Model = "Random Forest")

# KNN metrics
knn_metrics <- final_knn_fit %>%
  collect_metrics() %>%
  mutate(Model = "K-Nearest Neighbours")

# Combine all metrics
all_metrics <- bind_rows(lm_metrics, rf_metrics, knn_metrics) %>%
  select(Model, everything())  # put Model column first

print(all_metrics)
```

```
# A tibble: 8 × 5
  Model                .metric .estimator .estimate .config
  <chr>                <chr>   <chr>          <dbl> <chr>
1 Linear Model         rmse    standard       0.459 <NA>
2 Linear Model         rsq     standard       1.00  <NA>
3 Linear Model         mae     standard       0.367 <NA>
4 Random Forest        rmse    standard       2.63  <NA>
5 Random Forest        rsq     standard       0.975 <NA>
6 Random Forest        mae     standard       1.74  <NA>
7 K-Nearest Neighbours rmse    standard      10.5   Preprocessor1_Model1
8 K-Nearest Neighbours rsq     standard       0.603 Preprocessor1_Model1
```

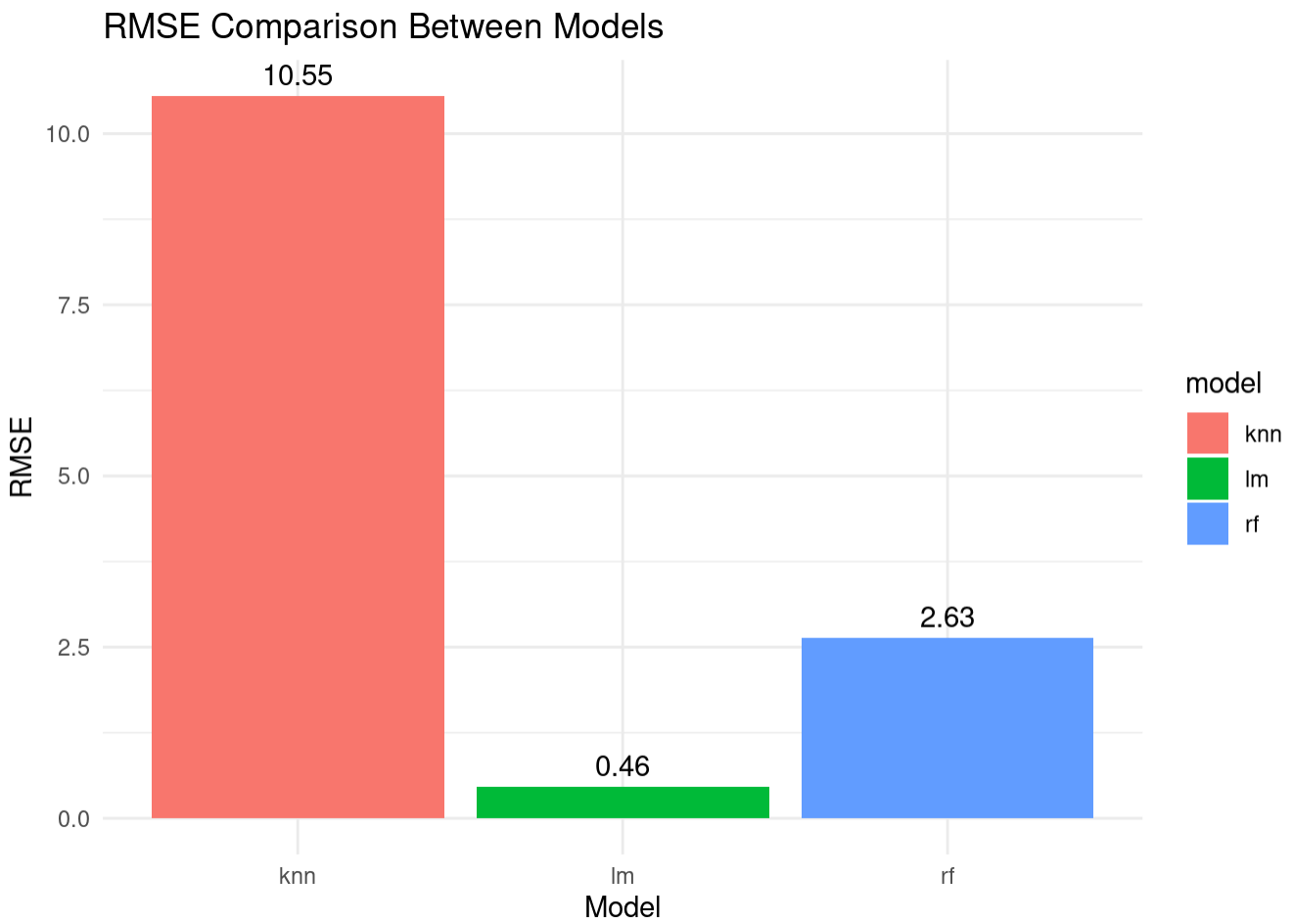
The table above shows the RMSE and RSQ metrics of the 3 models used in this report.
RMSE is highest in Linear regression and lowest in KNN, this suggests that there is minimal residual error in linear regression and KNN showed the highest residual error.
R square is approximately 1 in linear regression and is furthest from 1 in KNN.
This shows that linear regression has the best predictive performance.

```
# Evaluate the differences (metrics) between the models
compare_metrics <- bind_rows(
  bind_cols(
    lm_wf_prediction %>%
      metrics(truth = bone_mineral_density_score, estimate = .pred) %>%
      select(.metric, .estimate),
    model = "lm"
  ),
  bind_cols(
    rf_wf_prediction %>%
      metrics(truth = bone_mineral_density_score, estimate = .pred) %>%
      select(.metric, .estimate),
    model = "rf"
  ),
  bind_cols(
    final_knn_fit %>%
      collect_metrics() %>%
```

```
      collect_metrics() %>%
      select(.metric, .estimate),
    model = "knn"
  )
) %>%
  pivot_wider(
    names_from = .metric,
    values_from = .estimate
  )
```

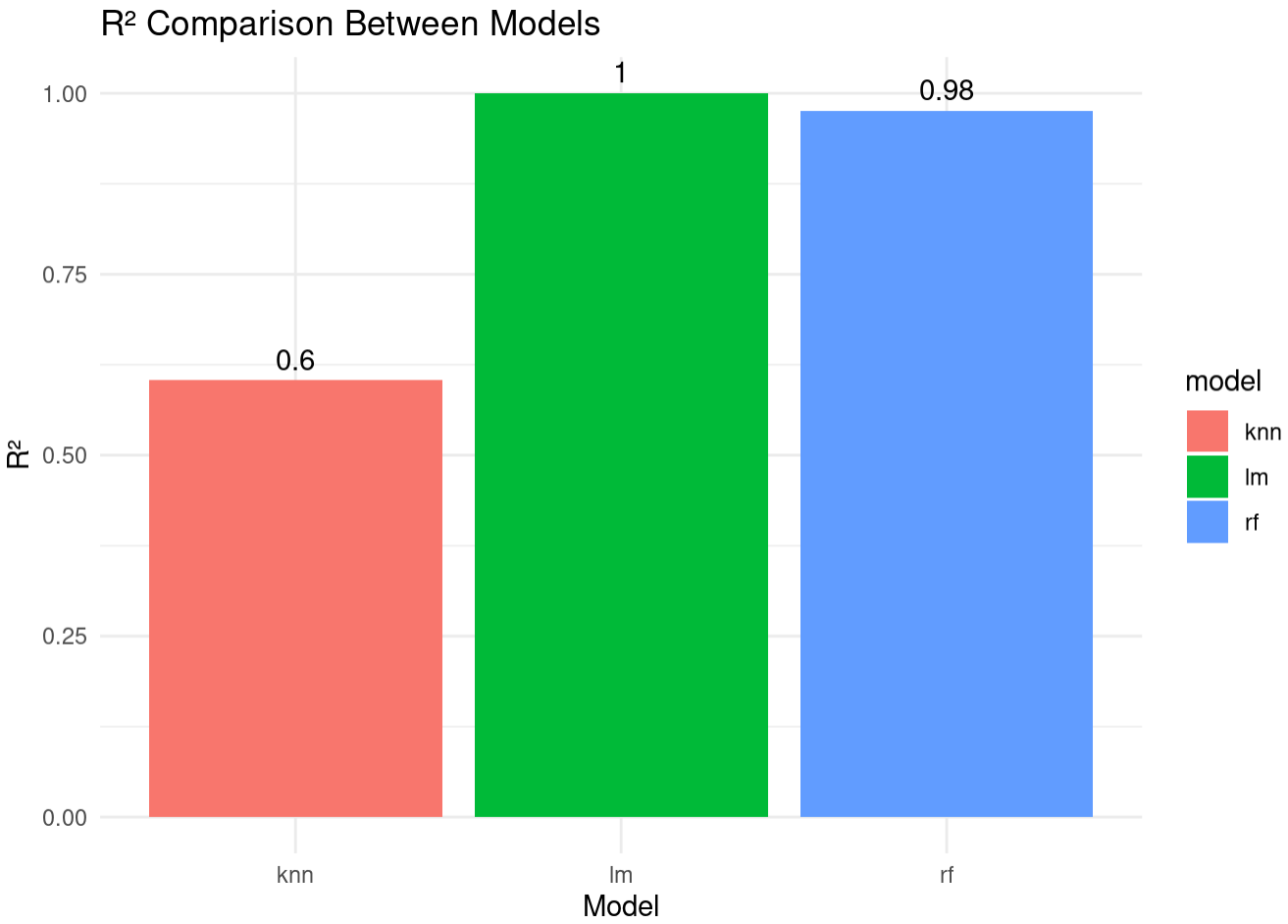
```
library(ggplot2)

# RMSE comparison
ggplot(compare_metrics, aes(x = model, y = rmse, fill = model)) +
  geom_col() +
  geom_text(aes(label = round(rmse, 2)), vjust = -0.5) +
  labs(title = "RMSE Comparison Between Models",
       x = "Model",
       y = "RMSE") +
  theme_minimal()
```




```
# R² (RSQ) comparison
ggplot(compare_metrics, aes(x = model, y = rsq, fill = model)) +
```

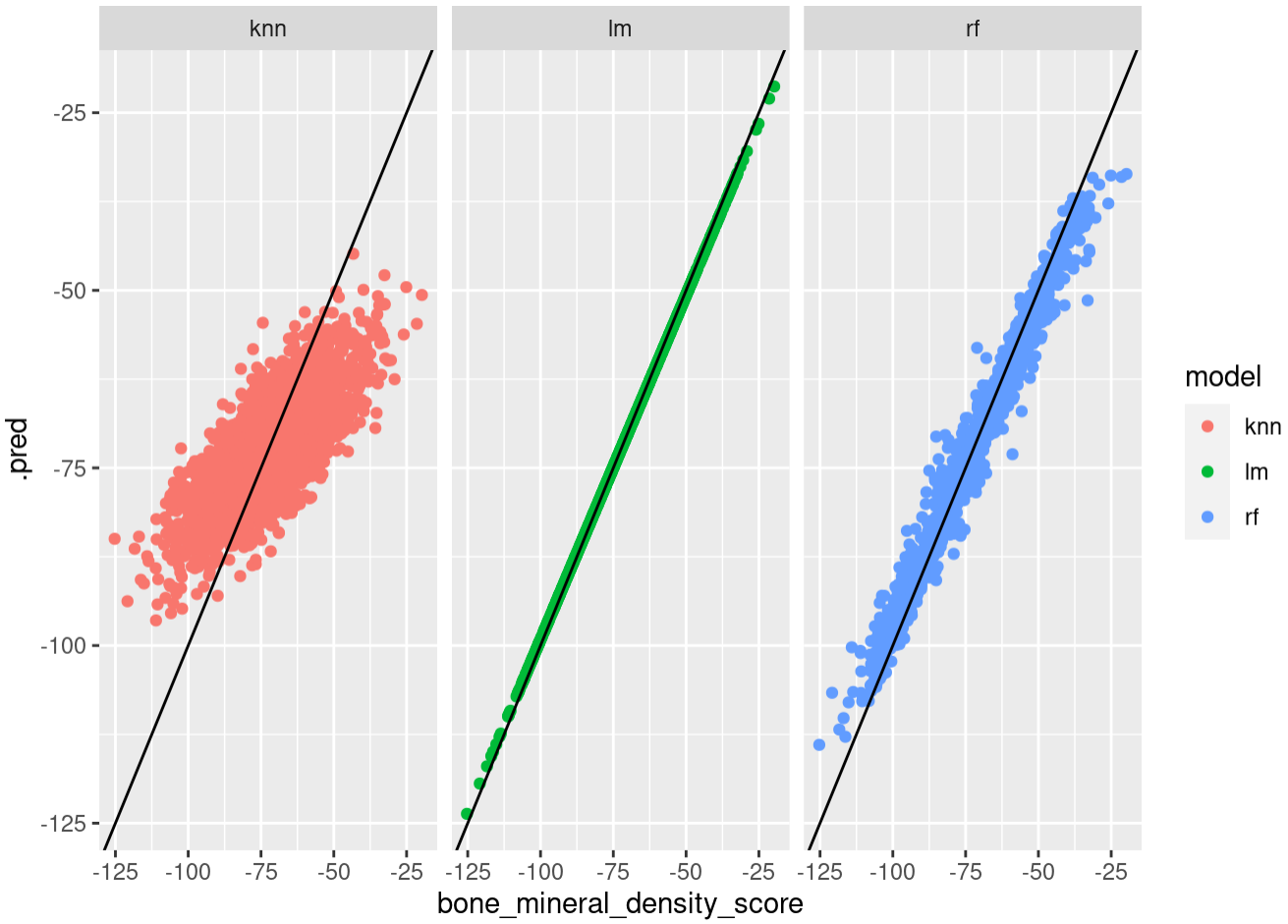
```
  geom_col() +
  geom_text(aes(label = round(rsq, 2)), vjust = -0.5) +
  labs(title = "R² Comparison Between Models",
       x = "Model",
       y = "R²") +
  theme_minimal()
```




```
# Compare the predictions between the models
compare_predictions <- bind_rows(
  bind_cols(
    lm_wf_prediction %>% select(bone_mineral_density_score, .pred),
    model = "lm"
  ),
  bind_cols(
    rf_wf_prediction %>% select(bone_mineral_density_score, .pred),
    model = "rf"
  ),
  bind_cols(
    final_knn_fit %>%
      collect_predictions(),
      model = "knn"
  )
)
```

```
,
```

```
# Plot the models predictions side to side
ggplot(compare_predictions, aes(x = bone_mineral_density_score, y = .pred, colour = model))+
  geom_point()+
  geom_abline()+
  facet_wrap(~model)
```



According to the plots, linear regression shows the best prediction performance, since almost all prediction values matches the outcome variable. Also random forest demonstrate strong predictive capabilities but less performance compared to linear regression. KNN model shows the lowest performance in terms of predictive accuracy.
Thus linear regression is recommended for the analysis of this specific data set.

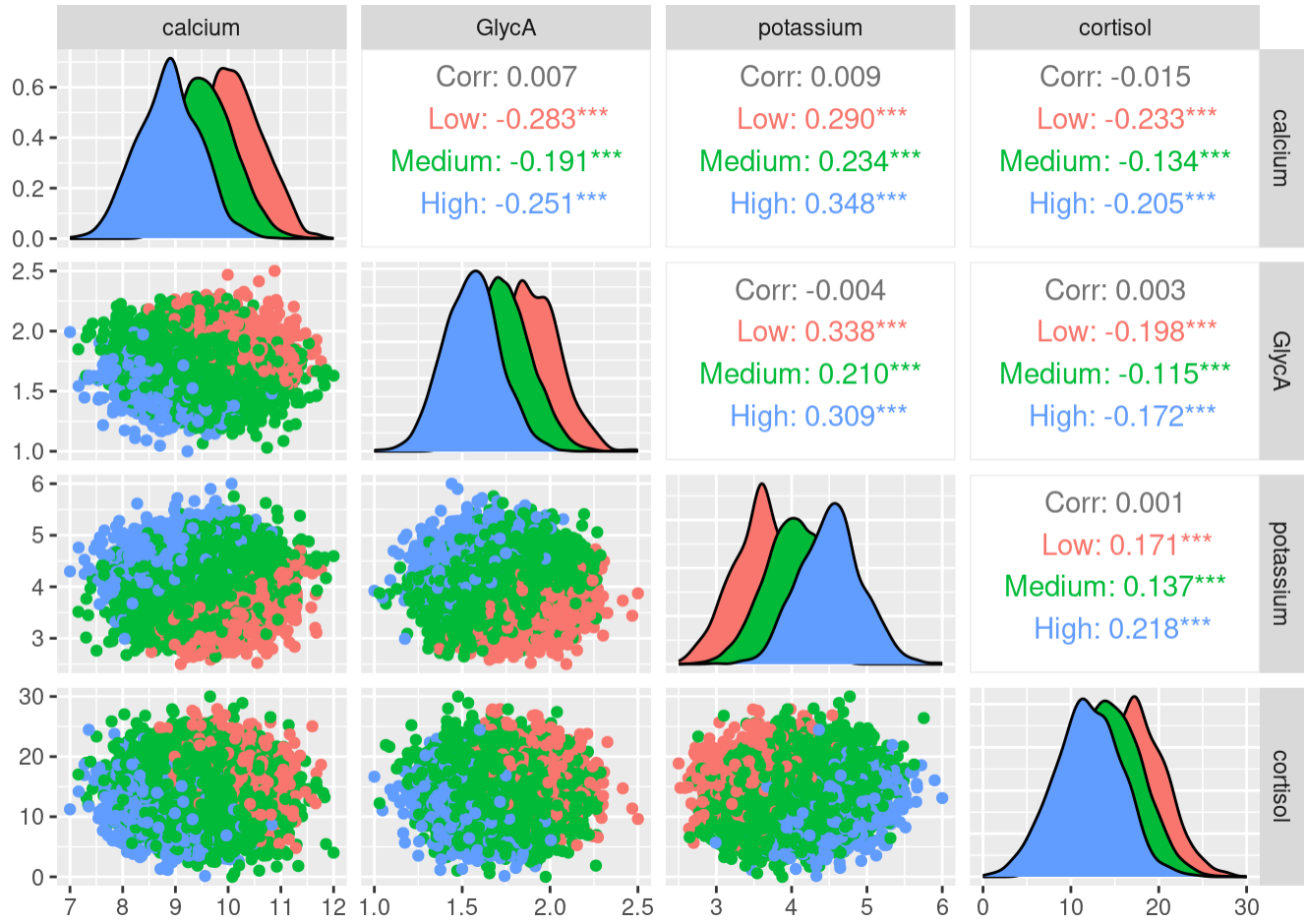
Studying the relationship between the 4 most important predictors and the outcome variable.

```
library(dplyr)
library(GGally)

# Convert numeric outcome to categorical
bone_density_data <- bone_density_data %>%
```

```
bone_density_data <- bone_density_data %>%
  mutate(bmd_category = cut(bone_mineral_density_score,
                            breaks = 3,  # 3 categories: Low / Medium / High
                            labels = c("Low", "Medium", "High")))

# Generate pair plot with categorical color
ggpairs(
  bone_density_data,
  columns = c(1, 19, 29, 30),   # indices of the 4 predictors
  aes(colour = bmd_category)
)
```



Overall the predictors are weakly correlated across the data set due to the points largely overlapping in the scatter plot and showing no trends.
The diagonal plots is a density plot that shows the distribution of the predictor split by the outcome category(red for low/green for medium/blue for high).
Similar trend are observed in calcium/GlycA/Cortisol, showing that for lower values for those predictors there is high bone_mineral_density_score and for higher values of those predictors there is low bone mineral density score. And the case is opposite for the potassium predictor. The upper triangle shows correlation coefficients between the two predictors. Over all the correlations between predictors is near 0.